

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software and Automation Engineering

Oberst Eduard, FAF-243

Report

Laboratory work nr. 2

Meet the OS You Will Build

xv6

Chișinău – 2026

Setup and objective

The objective of this laboratory work was to build and boot xv6, use its Unix utilities, read the user and kernel source code, and add a user program that waits for a specified number of clock ticks. The sleep program compiled successfully and ran inside xv6.

I used Ubuntu 26.04.1 LTS directly as the host. A separate Linux virtual machine was unnecessary. QEMU remained necessary to emulate the RISC-V machine on which xv6 runs. The repository and the required tools were already available.

```
$ cd ~/xv6-riscv
$ command -v riscv64-linux-gnu-gcc
/usr/bin/riscv64-linux-gnu-gcc
$ command -v qemu-system-riscv64
/usr/bin/qemu-system-riscv64
$ git log -1 --oneline
06aad25 Add killzero test
```

The installed cross-compiler was GCC 15.2.0, and the emulator was QEMU 10.2.1. The source tree separates privileged operating-system code in kernel/ from application code in user/.

Part 1 Build and boot

```
$ make qemu
[Compilation and filesystem build output omitted]
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

The build compiled user/sleep.c and linked user/_sleep. The filesystem image was rebuilt with the standard utilities and the new sleep program. The boot messages and shell prompt confirmed that xv6 started successfully. To leave QEMU, I pressed Ctrl-A, released it, and pressed X.

Part 2 Using xv6

I ran the commands below at the xv6 shell prompt. Long outputs are shown as excerpts; the commands and values come from the verified run.

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
[Other entries omitted]
sleep          2 23 35496
console        3 24 0
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and
Ken Thompson's Unix Version 6 (v6).
[Remaining README text omitted]
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 24 0
$ wc README
48 336 2441 README
```

Observation answers

1. Three programs shipped with xv6 are ls, cat, and echo. ls lists directory entries, cat copies file contents to standard output, and echo prints its arguments.

2. A pipe requires process support and inter-process communication. The shell creates processes for the two commands and connects the producer’s standard output to the consumer’s standard input through pipe file descriptors.

3. The xv6 shell supports familiar Unix commands, redirection, and pipes, but provides fewer features than the Linux shell used in Lab 1.

The optional usertests -q suite was not run; verification covered booting, the required utilities, piping, and the new sleep program.

Part 3 Reading the source

I inspected user/ls.c, user/cat.c, user/user.h, and kernel/sysfile.c. The ls program opens a path and uses fstat to inspect it. The cat program reads data into a 512-byte buffer and writes that data to standard output.

```
/* Excerpt from user/cat.c */
while ((n = read(fd, buf, sizeof(buf))) > 0) {
  if (write(1, buf, n) != n) {
    fprintf(2, "cat: write error\n");
    exit(1);
  }
}
```

System call declarations and handlers

```
/* Declarations in user/user.h */
int read(int, void *, int);
int write(int, const void *, int);
int pause(int);

/* Function locations in kernel/sysfile.c */
69:sys_read(void)
83:sys_write(void)
```

The declarations provide the user API. Generated system-call stubs use the RISC-V ecall instruction to enter the kernel. The kernel dispatches the request to the corresponding handler; sys_read ultimately calls fileread, and sys_write calls filewrite.

Observation answers

1. user/cat.c uses open(), read(), write(), close(), and exit(). open() asks the kernel to open a file and return a file descriptor. read() requests bytes from a descriptor. write() sends bytes to a descriptor, including standard output (1). close() releases the descriptor. exit() terminates the process with a status. fprintf() is a user-library function that eventually uses write(), rather than a separate system call.

2. sys_read is implemented in kernel/sysfile.c. In revision 06aad25, its function name appears on line 69; the uint64 return type is on line 68.

3. kernel/ contains privileged code that manages hardware and operating-system resources, while user/ contains ordinary applications that request kernel services through system calls.

Part 4 Writing the sleep program

The assignment calls the delay system call sleep(). In this xv6 revision, that API is named pause(), with the kernel handler sys_pause(). I used a small sleep() wrapper around pause() so the user command keeps the required name and waits for clock ticks without changing the kernel.

```
#include "kernel/types.h"
#include "user/user.h"

// This xv6 version names the tick-delay system call pause().
static int
sleep(int ticks)
{
  return pause(ticks);
}

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  sleep(atoi(argv[1]));
  exit(0);
}
```

Registering the program in the build

I added the following entry to the Makefile UPROGS list. The entry uses a tab for indentation and a trailing backslash to continue the list.

```
UPROGS=\
    [Existing entries omitted]
    $U/_sync\
    $U/_sleep\
```

The actual Makefile uses tabs in front of the program entries. Running make qemu then compiled and linked the program and included it in fs.img. argc checks that exactly one argument is supplied, atoi converts it to ticks, and exit(0) reports successful completion.

Testing and results

I tested the sleep command inside the running xv6 system after rebuilding. The successful case waited and returned without output. Both invalid argument counts produced the required usage message.

```
$ sleep 10
$ echo sleep-test-complete
sleep-test-complete
$ sleep
usage: sleep <ticks>
$ sleep 10 20
usage: sleep <ticks>
$
```

The shell prompt returned after sleep 10, and the following echo command executed. In the missing-argument and extra-argument cases, the program exits with status 1 as specified by the source code. The successful path exits with status 0.

Submitted files

user/sleep.c contains the working user program. notes.md contains the answers for Parts 2 and 3 and the verification results. The Makefile change is also required to build and include the program. These files are prepared locally in ~/xv6-riscv.

Conclusion

In this laboratory work, I built and booted xv6 on emulated RISC-V hardware from my Ubuntu host, used its standard Unix utilities, and inspected the boundary between applications and the kernel. Reading cat and its system-call handlers connected user-level file operations to the kernel code that implements them.

I added the sleep user program to the filesystem build and verified its successful delay and usage checks. Adapting the call to pause() matched the API of the installed xv6 revision. The work established the build, boot, source-reading, and testing workflow needed for later kernel exercises.